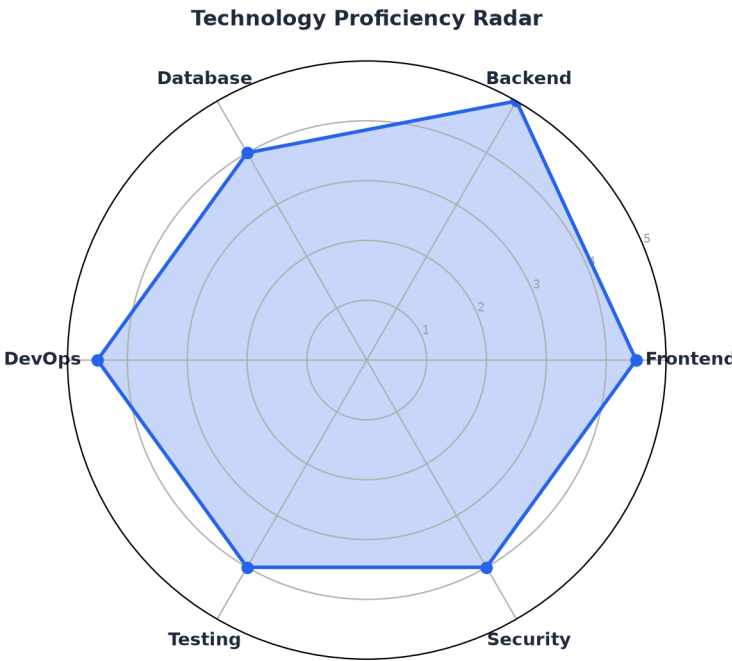


Quini6 Analytics

Documento Tecnico

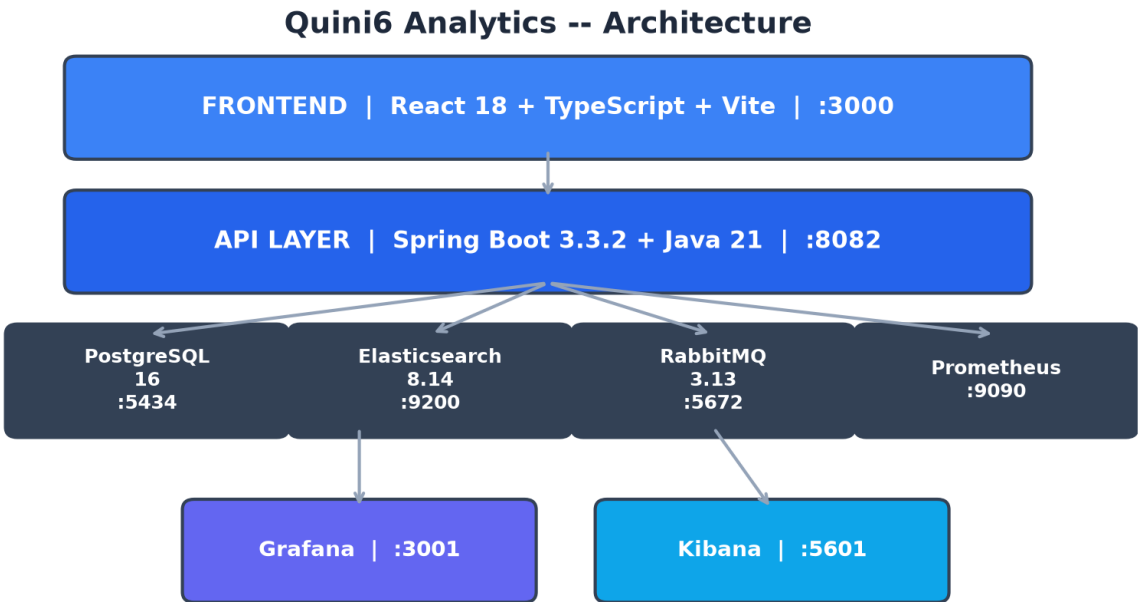
Arquitectura de Software - Entrevista Tecnica



Java 21 | Spring Boot 3.3 | React 18 | PostgreSQL | Docker
Los sorteos del Quini 6 son eventos independientes (i.i.d.)

1. Arquitectura General

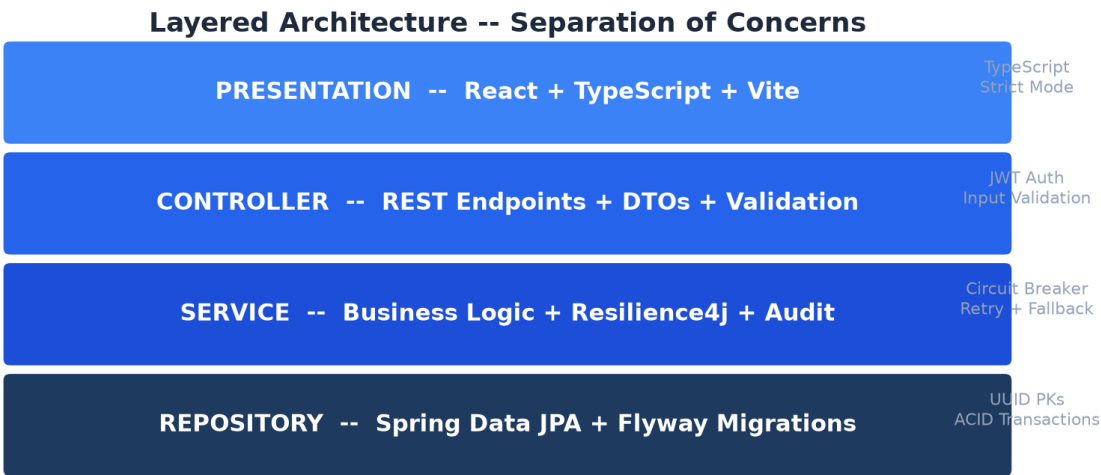
Quini6 Analytics sigue un patron de arquitectura por capas (Layered Architecture) con separacion clara de responsabilidades. El sistema esta dividido en: Presentacion (React), Controladores (REST API), Servicios (logica de negocio), y Repositorios (persistencia).



Decision de Diseno: Monolito Modular

Se eligio un monolito modular en lugar de microservicios por tres razones: (1) La escala actual no justifica la complejidad de red y distribucion. (2) La separacion por paquetes permite extraccion futura a microservicios. (3) El despliegue y mantenimiento son significativamente mas simples.

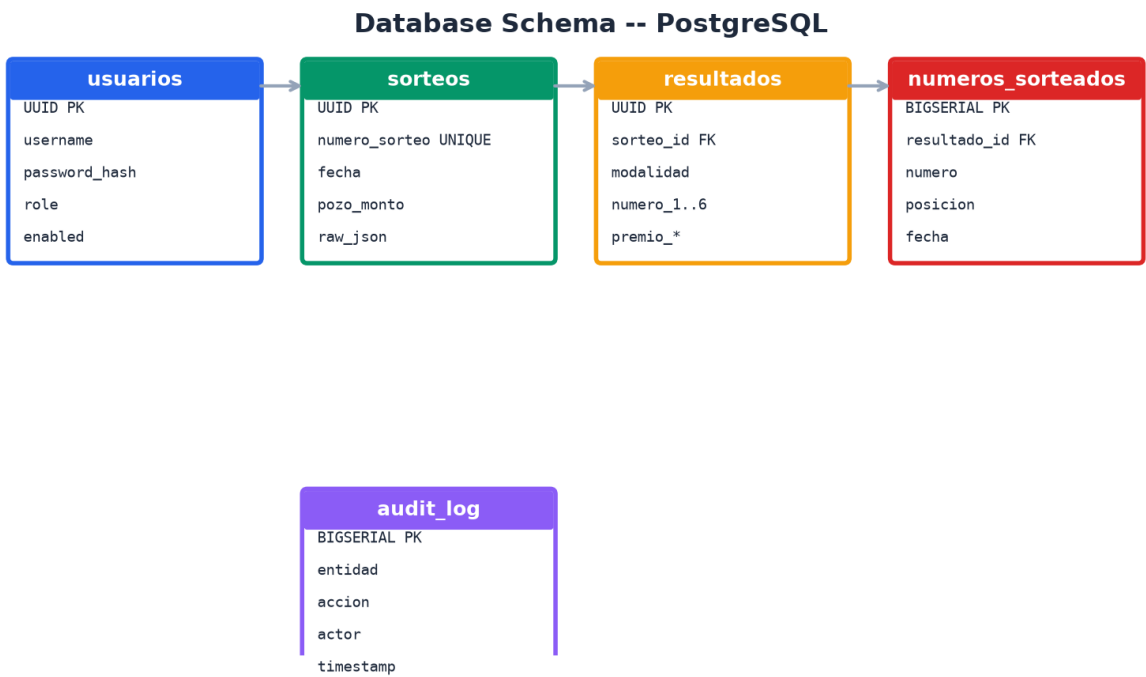
2. Capas de la Arquitectura



Patrones Aplicados

Patron	Implementacion	Beneficio
Repository	Spring Data JPA	Abstraccion de persistencia
DTO	Records Java	Inmutabilidad, sin boilerplate
Circuit Breaker	Resilience4j	Graceful degradation
Publisher/Listener	RabbitMQ	Desacoplamiento async
AOP Audit	@Audit annotation	Trail de cambios
JWT Filter	OncePerRequestFilter	Auth stateless

3. Base de Datos



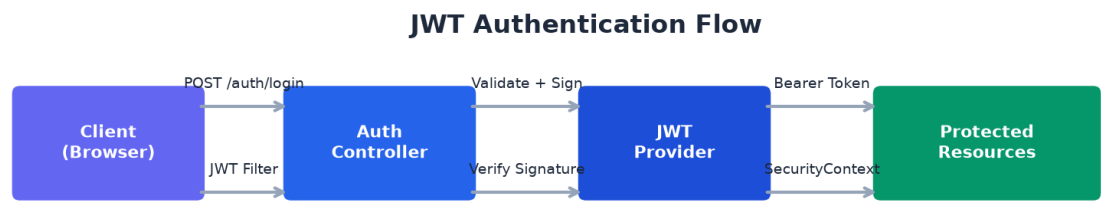
Los sorteos del Quini 6 son eventos independientes (i.i.d.). Analisis descriptivo solamente.

Decisiones de Modelado

- UUID como PK: Evita auto-increment expuesto, mejor para distribucion

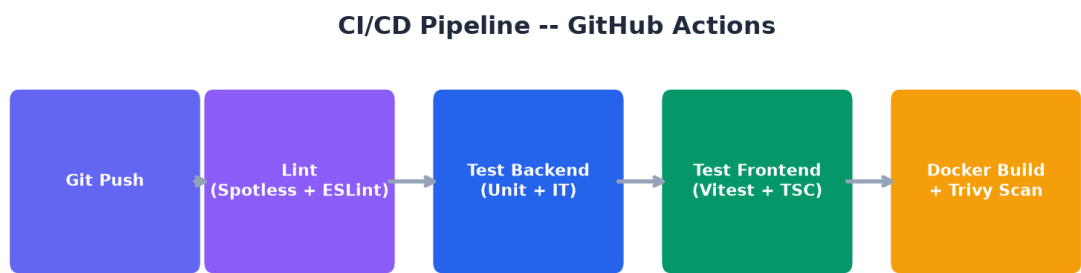
- Tal

4. Seguridad y Autenticacion

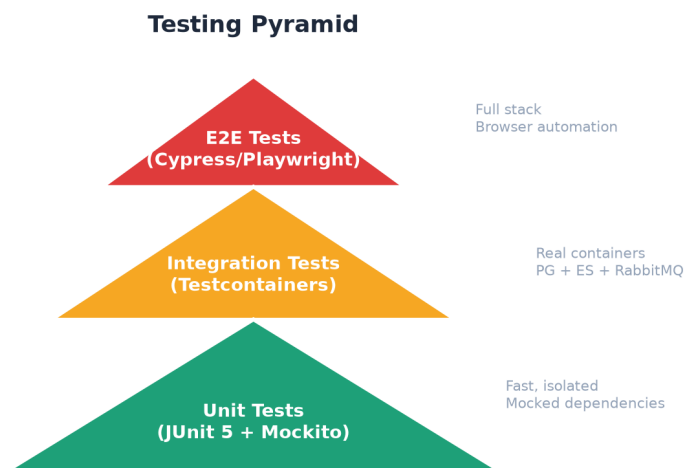


El sistema utiliza JWT (JSON Web Tokens) con HS256 para autenticacion stateless. El password se almacena con BCrypt. El filtro JWT se ejecuta en cada request para validar token y establecer SecurityContext.

5. CI/CD y Testing



Testing Pyramid



La estrategia de testing sigue la piramide: muchos unit tests (rapidos, aislados), algunos integration tests (con Testcontainers para PG, ES, RabbitMQ), y pocos E2E tests (futuro). Cobertura minima: 75% en servicios de negocio.

6. Trade-offs y Decisiones Tecnicas

Alternativa Rechazada	Razon
Microservicios	Complejidad prematura para la escala actual
MongoDB	Relaciones complejas requieren ACID
Redis como DB	Cache, no almacenamiento persistente
GraphQL	REST es suficiente y mas simple
JOOM / Exposed	JPA tiene mas comunidad en ecosistema Spring
Webpack	Vite es mas rapido por ESM nativo

7. Lecciones Aprendidas

- TypeScript evita bugs que serian runtime errors en JavaScript puro

- El

8. Conclusion

Este proyecto demuestra la aplicacion de patrones de arquitectura de software en un caso real: capas bien definidas, separacion de responsabilidades, observabilidad completa, y un pipeline de CI/CD que garantiza calidad. Los conceptos aplicados -- JWT, circuit breakers, CQRS ligero, event-driven architecture, infrastructure as code -- son directamente transferibles a cualquier proyecto empresarial.